

ENTREGABLE PROYECTO DE CURSO TERCER CORTE



Universidad
del Cauca

Elaborado por:

IIKAN YOHARY BONILLA ARIAS

JUAN SEBASTIAN CERON PAZ

SARAH ANDREA QUINTERO GOMEZ

ANDREA FERNANDA GOMEZ SALAZAR

CARLOS ANDRES CAICEDO DAZA

Universidad del Cauca

Facultad de Ingeniería Electrónica y Telecomunicaciones

Ingeniería de Sistemas

Software II

Popayán, Cauca, Colombia

TABLA DE CONTENIDOS

INTRODUCCIÓN BREVE.....	3
HISTORIAS DE USUARIO.....	4
Historias de Usuario – Iteración 1.....	4
Historias de Usuario – Iteración 2.....	6
Sprint 3 (3er corte: requisitos 3, 4, 5 y 6) – Iteración 3.....	10
PLANIFICACIÓN.....	12
Planificación De Tareas Del Sprint 1.....	12
Planificación De Tareas Del Sprint 2.....	13
Planificación De Tareas Del Sprint 3.....	14
ESCENARIOS DE CALIDAD.....	15
Escenario 1 - Modificabilidad.....	15
Escenario 2 - Escalabilidad.....	15
ARQUITECTURA Y DISEÑO.....	17
ITERACIÓN 2.....	17
Diagrama de Contexto (Nivel 1 C4).....	17
Diagrama de Contenedores (Nivel 2 C4).....	18
Diagrama de Componentes (Nivel 3 C4).....	19
Diagrama de Bounded Contexts.....	21
Diagrama de Clases UML.....	21
ITERACIÓN 3.....	24
Diagrama de Contexto (Nivel 1 C4).....	24
Diagrama de Contenedores (Nivel 2 C4).....	25
Diagrama de Componentes (Nivel 3 C4) - booking-service.....	26
Diagrama de Bounded Contexts.....	29
Diagrama de Clases UML.....	32
Aspectos de seguridad evidenciados en C4.....	37
PATRONES DE DISEÑO GOF.....	38
Listado De Patrones De Diseño Implementados.....	38
REPOSITORIO GIT.....	40

INTRODUCCIÓN BREVE

El presente documento expone la arquitectura definitiva del Sistema de Reserva de Citas de Piedrazul, consolidado a lo largo de tres iteraciones académicas en la asignatura de Ingeniería de Software II (2026.1). Esta solución tecnológica faculta a agendadores, pacientes, profesionales de la salud y administradores para la gestión de citas médicas bajo criterios de seguridad, usabilidad y escalabilidad.

La propuesta técnica transitó desde los requerimientos iniciales de primer corte (agendamiento vía WhatsApp) hacia una arquitectura orientada a microservicios con Spring Boot 3.2, un portal web en Angular 21, persistencia en PostgreSQL aislada por servicio y mensajería asíncrona a través de RabbitMQ. Durante el tercer corte, se integran funcionalidades de agendamiento autónomo, gestión administrativa de disponibilidad, exportación de datos en formato CSV y trazabilidad histórica en el re-agendamiento.

Este entregable detalla las historias de usuario por iteración, la planificación de los sprints realizados, los escenarios de calidad enfocados en modificabilidad y escalabilidad, diagramas de modelo C4 con énfasis en seguridad para las fases finales y el catálogo de patrones de diseño GoF aplicados en el desarrollo del software.

HISTORIAS DE USUARIO

Historias de Usuario – Iteración 1

Historias de USUARIO y criterios de aceptación de HE-02

Identificador (ID) de la épica	Identificador (ID) de la historia	Enunciado de la historia				Criterios de aceptación			
		Rol	Característica / Funcionalidad (Quiero)	Razón / Resultado (Para)	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
	HU-1	Yo como USUARIO	Necesito ingresar al sistema.	Para poder hacer uso de las funcionalidades.	1	Ingreso Exitoso.	En caso de ingresar el USUARIO y la clave correcta.	Clic en el boton ENVIAR	El sistema despliega la pagina de inicio con las opciones del ROL del USUARIO.
					2	Ingreso Fallido.	En caso de ingresar el USUARIO o la clave incorrecta.	Clic en el boton ENVIAR	El sistema muestra el mensaje: 'USUARIO o CLAVE incorrecta'.
					3	Multiples Ingresos fallidos	En caso de ingresar el USUARIO o la clave incorrecta diez veces dentro de un periodo de tiempo de una hora.	Clic en el boton ENVIAR	El sistema muestra el mensaje: 'Ha ingresado multiples veces informacion erronea, se ha restringido el ingreso desde este equipo, intentelo nuevamente dentro de una hora'.
					4	Falta de ingreso de campo obligatorio.	En caso de que NO se ingrese el USUARIO o la clave.	Clic en el boton ENVIAR	El sistema muestra el mensaje: 'Ambos Campos son Obligatorios'.
					5	Recuperación de Clave	En caso de olvidar mi clave de acceso.	Clic en el enlace '¿Has olvidado tu clave?'	El sistema despliega la pagina de recuperación de clave. (HU-1.1)
	HU-1.1	Yo como USUARIO	Necesito recuperar mi clave de acceso al sistema.	Para poder ingresar al sistema.	1	Recuperación Exitosa.	En caso de ingresar un nombre de USUARIO reconocido por el sistema.	Clic en el boton RECUPERAR	El sistema muestra el mensaje: 'Se ha enviado un correo para la recuperacion de su clave'.
					2	USUARIO no reconocido.	En caso de ingresar un nombre de USUARIO no reconocido por el sistema.	Clic en el boton RECUPERAR	El sistema muestra el mensaje: 'Se ha enviado un correo para la recuperacion de su clave'.
					3	Falta de ingreso de campo obligatorio.	En caso de que NO se ingrese el USUARIO.	Clic en el boton RECUPERAR	El sistema muestra el mensaje: 'El ingreso del USUARIO es necesario'.
					4	Regreso a pagina de Inicio de Sesión.	En caso de querer regresar a la pagina de ingreso.	Clic en el boton VOLVER	El sistema despliega la pagina de Inicio de Sesión. (HU-1)

Historias de USUARIO y criterios de aceptación de HE-06

Identificador (ID) de la historia	Identificador (ID) de la historia	Enunciado de la historia			Criterios de aceptación				
		Rol	Característica / Funcionalidad (Quiero)	Razón / Resultado (Para)	Número (#) de escenario	Criterio de aceptación (Título)	Contexto	Evento	Resultado / Comportamiento esperado
	HU-6	Yo como MÉDICO, TERAPISTA O AGENDADOR	Quiero registrar manualmente una cita	Para programar la atención de un paciente en una fecha y hora específica.	1	Creación exitosa.	En caso de ingresar los siguientes campos Obligatorios Correctamente : DOCUMENTO DE IDENTIDAD DEL PACIENTE, NOMBRES Y APELLIDOS, NÚMERO DE TELÉFONO, GÉNERO, HORA, FECHA,	Clic en el boton GUARDAR	El sistema despliega la página de TUS CITAS ACTUALES
					2	Creación Fallida.	En caso de ingresar : DOCUMENTO DE IDENTIDAD DEL PACIENTE no válido. (menos de 5 caracteres, caracteres no numéricos) NÚMERO DE TELÉFONO no válido (cantidad de caracteres diferente de 10) GÉNERO no válido (Diferente a las opciones dadas), HORA no válido.	Clic en el boton GUARDAR	El sistema muestra la advertencia: 'No se ha podido AGENDAR TU CITA' resaltando el campo no valido y la razon por la que no lo es.
					3	Campos vacíos.	En caso de dejar vacío alguno de los siguientes campos Obligatorios : DOCUMENTO DE IDENTIDAD DEL PACIENTE, NOMBRES Y APELLIDOS,	Clic en el boton GUARDAR	El sistema muestra la advertencia: 'El campo el obligatorio' resaltando el campo vacío.
							NÚMERO DE TELÉFONO, GÉNERO, HORA, FECHA,		
					4	Listado de Citas.	En caso de haber ingresado al listado de MIS CITAS	Clic en la Botón: VER CITAS	El sistema despliega un listado de las citas del usuario actual y las acciones: NUEVA CITA, REAGENDAR CITA (HU-6.1), EXPORTAR (HU-6.2)
					5	Cancelar Agendamiento.	En caso de dar clic en el boton CANCELAR. En el formulario de AGENDAR NUEVA CITA	Clic en el boton CANCELAR.	El sistema redirige al listado de citas MIS CITAS

HE-06	HU-6.1	Yo como MÉDICO, TERAPISTA O AGENDADOR	Quiero modificar la fecha y la hora de una cita existente.	Para reprogramar la atención cuando sea necesario.	1	Reagendamiento exitoso.	En caso de ingresar correctamente la nueva información en los campos: HORA, FECHA. En el formulario REAGENDAR CITA.	Clic en el boton GUARDAR	El sistema despliega la página de TUS CITAS ACTUALES
					2	Reagendamiento fallido.	En caso de ingresar: FECHA no válido. (Fecha de atención no disponible) HORA no válido. (Hora y fecha pertenecen a una cita existente) En el formulario REAGENDAR CITA.	Clic en el boton GUARDAR	El sistema mostrará las fechas y hora disponibles para esa semana, en caso de ser viernes mostrará la siguiente semana.
					3	Cancelar Reagendamiento.	En caso de dar clic en el boton CANCELAR. En el formulario de REAGENDAR CITA.	Clic en el boton CANCELAR.	El sistema redirige al listado de citas MIS CITAS
	HU-6.2	Yo como MÉDICO, TERAPISTA O AGENDADOR	Quiero exportar las citas registradas a un archivo CSV	Para analizarlas o compartir la información externamente	1	Range.	En caso de haber ingresado en el formulario CITAS A EXPORTAR	Clic en el boton EXPORTAR.	El sistema permite ver el las opciones disponibles: HOY VARIOS (LUNES MARTES MIERCOLES JUEVES VIERNES) Y los botones: DESCARGAR CANCELAR
					2	Descargar archivo exitoso.	En caso de seleccionar el boton DESCARGAR. En la vista CITAS A EXPORTAR.	Clic en el boton DESCARGAR.	El sistema descarga el archivo en el dispositivo.
					3	Descargar archivo fallido.	En caso de ingresar: VARIOS no válido. (Selección de días mayor a 3)	Clic en el botón DESCARGAR	El sistema enviará un mensaje de error: 'NO se pueden seleccionar más de 3 días al exportar las citas' Mostrando el número de citas seleccionadas 'Número de citas: (Número)'
					4	Cancelar CSV.	En caso de seleccionar el boton CANCELAR. En la vista CITAS A EXPORTAR.	Clic en el boton CANCELAR.	El sistema dirige al listado de citas MIS CITAS.

Historias de Usuario – Iteración 2

- Historia 1 – Crear usuario (Administrador)

Criterios de aceptación:

- Datos obligatorios: nombre, documento, rol, correo, contraseña.
- Validar que el usuario no exista previamente.
- Confirmar creación del usuario.

Complemento arquitectónico:

- Se implementa con Spring Boot (MVC) para modularidad y seguridad.
- El Security Module (Spring Security) garantiza roles y autenticación.
- La información persiste en Postgres, con auditoría de cambios.

Escenario de calidad aplicado:

- Seguridad: acceso restringido según rol, registro de intentos fallidos.

- Historia 2 – Editar usuario (Administrador)

Criterios de aceptación:

- Modificar datos personales.No duplicar correo o documento.Guardar registro de cambios.

Complemento arquitectónico:

- UsuarioService gestiona la lógica de negocio.
- Cambios registrados en la entidad Auditoría.
- Persistencia en MySQL con validaciones de integridad.

- Historia 3 – Consultar usuarios (Administrador)

Criterios de aceptación:

- Mostrar tabla con usuarios.
- Búsqueda por nombre o rol.
- Visualizar el estado del usuario.

Complemento arquitectónico:

- Angular en el frontend para una interfaz visual moderna.
- Spring Boot REST API expone endpoints para consulta.
- POSTGRES almacena usuarios y roles.

- Historia 4 – Desactivar usuario (Administrador)

Criterios de aceptación:

- Usuario desactivado no puede iniciar sesión.
- Mantener información histórica.

Complemento arquitectónico:

- Security Module valida estado del usuario.
- Persistencia en POSTGRES mantiene registros históricos.

- Historia 5 – Inicio de sesión (Usuario)

Criterios de aceptación:

- Validar usuario y contraseña.
- Mensaje si las credenciales son incorrectas.
- Redirigir al panel según rol.

Complemento arquitectónico:

- Spring Security gestiona autenticación y roles.
- Contraseñas cifradas en la base de datos.

Escenario de calidad aplicado:

- Seguridad: bloqueo tras intentos fallidos, trazabilidad en auditoría.

- Historia 6 – Recuperación de contraseña (Usuario)

Criterios de aceptación:

- Ingresar correo electrónico.
- Enviar enlace de recuperación.
- Permitir cambio de contraseña.

Complemento arquitectónico:

- NotificationService envía correos electrónicos.
- Spring Boot gestiona flujo seguro de recuperación.

- Historia 7 – Control de roles (Sistema)

Criterios de aceptación:

- Administrador: acceso completo.
- Médico/Terapista: acceso a citas e historia clínica.
- Paciente: acceso solo a sus citas.

Complemento arquitectónico:

- Spring Security implementa control de roles.
- Auditoría registra accesos y acciones.

Escenario de calidad aplicado:

- Seguridad: acceso restringido, trazabilidad completa.

- Historia 8 – Agendar cita (Paciente)

Criterios de aceptación:

- Usuario registrado.
- Mostrar franjas disponibles.
- Confirmar cita.

Complemento arquitectónico:

- CitaService gestiona la lógica de negocio.
- Angular ofrece una interfaz intuitiva y compatible con API REST.
- Postgres almacena citas con relación a paciente y profesional.

Escenario de calidad aplicado:

- Usabilidad: flujo simple, confirmación visual, 90% de usuarios completan en <4 min.

- Historia 9 – Agendamiento autónomo vía web (Paciente)

Criterios de aceptación:

- El paciente debe estar registrado en el sistema.
- El sistema debe mostrar las franjas horarias disponibles por médico/terapista.
- El flujo de agendamiento debe ser simple e intuitivo (selección de especialidad, profesional, fecha y hora).
- Confirmación visual y notificación al paciente (correo/WhatsApp).

Complemento arquitectónico:

- Frontend Angular ofrece interfaz intuitiva.
- CitaService gestiona la lógica de negocio y disponibilidad.
- NotificacionService envía confirmaciones.
- Persistencia en Postgres con relación a paciente y profesional.

Escenario de calidad aplicado:

- Usabilidad: al menos el 90 % de usuarios nuevos debe poder agendar en menos de 4 minutos, con máximo un error leve y sin capacitación previa.

- Historia 10 – Agendamiento autónomo vía web (Paciente)

Criterios de aceptación:

- Configurar ventana de tiempo habilitada para citas (en semanas).
- Definir días de atención de cada médico/terapista.
- Establecer franjas horarias e intervalos entre citas.
- Guardar y aplicar cambios de forma inmediata.

Complemento arquitectónico:

- ConfiguracionService gestiona parámetros globales.
- Singleton asegura que la configuración sea única en todo el sistema.
- Persistencia en POSTGRES para mantener parámetros históricos.
- Auditoría registra cambios realizados por el administrador.

Escenario de calidad aplicado:

- Escalabilidad: la configuración flexible permite que el sistema se adapte a cambios en disponibilidad sin afectar la estabilidad.
- Seguridad: solo administradores autenticados pueden modificar parámetros, con registro de auditoría.

Sprint 3 (3er corte: requisitos 3, 4, 5 y 6) – Iteración 3

- Historia 6 – Agendar cita autónoma vía web (Paciente)

Criterios de aceptación:

- El paciente debe registrarse previamente (POST /users/web-register).
- Mostrar franjas disponibles por médico y fecha.
- Flujo simple: selección de médico, fecha, hora y confirmación.
- Validar que la franja siga disponible al confirmar.

Complemento arquitectónico:

- Facade WebBookingFacade orquesta disponibilidad + creación de cita en una sola operación.
- Endpoint POST /appointments/web-booking.
- Frontend en ruta /paciente.
- Validar que la franja siga disponible al confirmar.

Escenario de calidad aplicado:

- Usabilidad: flujo en menos de 4 minutos para usuarios nuevos.
- Seguridad: verificación de disponibilidad en servidor antes de persistir (evita doble reserva).

- Historia 7 – Configurar parámetros del sistema (Administrador)

Criterios de aceptación:

- Configurar ventana de agendamiento en semanas.
- Definir días de atención, franja horaria e intervalo por médico/terapeuta.
- Aplicar cambios de forma inmediata al consultar disponibilidad.

Complemento arquitectónico:

- SystemConfigController: PUT /system-config/booking-window.
- AvailabilityController: POST /availability/configure.
- Frontend en /admin/disponibilidad y /admin/medicos.

- Persistencia en availability_db.

Escenario de calidad aplicado:

- Escalabilidad: configuración flexible permite adaptar disponibilidad sin afectar estabilidad del sistema.
- Modificabilidad: parámetros centralizados en SystemConfigService.

- Historia 8 – Exportar citas a CSV (Médico/Agendador)

Criterios de aceptación:

- Seleccionar médico y fecha.
- Descargar archivo CSV compatible con hojas de cálculo.
- Incluir datos del paciente y hora de la cita.

Complemento arquitectónico:

- GET /appointments/doctor/{id}/export?date= retorna text/csv.
- AppointmentService.exportAppointmentsToCsv() genera el contenido.
- UserClientAdapter enriquece datos del paciente.
- Botón "Exportar CSV" en pantalla /agendador.

- Historia 9 – Re-agendar cita con historial (Médico/Terapista)

Criterios de aceptación:

- Seleccionar cita existente y nueva fecha/hora.
- Validar disponibilidad y evitar solapamiento.
- Conservar historial: fecha anterior, nueva fecha y responsable.
- No permitir re-agendar citas canceladas o completadas.

Complemento arquitectónico:

- Patrón Command: RescheduleAppointmentCommand, CommandInvoker, RescheduleAppointmentCommandHandler.
- Patrón State: AppointmentStateContext valida transiciones según estado.
- Entidad AppointmentRescheduleHistory en booking_db.
- Endpoints: PUT /appointments/{id}/reschedule y GET /appointments/{id}/reschedule-history.

Escenario de calidad aplicado:

- Modificabilidad: nuevos comandos (ej. cancelación masiva) se agregan sin modificar CommandInvoker.
- Seguridad: reglas de negocio encapsuladas en estados terminales (TerminalAppointmentState).

PLANIFICACIÓN

Planificación De Tareas Del Sprint 1



https://www.notion.so/32d27cfcba7806fa54dc98ae1cbd7f8?v=32d27cfcba780a3a1d0000ce568db89&source=copy_link

El Sprint 1 fue planificado utilizando un tablero Kanban en Notion, organizado en tres columnas: “Sin empezar”, “En progreso” y “Listo”.

Las tareas fueron definidas a partir de las historias de usuario priorizadas y descompuestas en actividades técnicas de frontend, backend y diseño.

Planificación De Tareas Del Sprint 2

Q Buscar

+ Crear

Mejorar versión

9+

3+

⚙️

SG

Espacio

Sistema Piedrazul

...

🔗

⚡

💬

↗️

🌐 Resumen

📅 Lista

📁 Tablero

📅 Backlog

📅 Calendario

📅 Cronograma

📄 Documentos

📄 Formularios

🔗 Desarrollo

+

Q Buscar ac...

👤

SG

AS

CD

🔍

Filtro

2

📁

Agrupar

Filtros guardados

📅

📁

...

☐	Actividad	Persona asignada	Informador	Prioridad	
☐	PIED1-29 Realizar pruebas unitarias	CD CARLOS ANDRES CA...	AS ANDREA FERNA...	🔥 Medium	LISTO
☐	PIED1-28 Aplicar configuración al sistema	AS ANDREA FERNANDA ...	AS ANDREA FERNA...	🔥 Medium	LISTO
☐	PIED1-27 Guardar configuración	SG SARAH ANDREA QUI...	AS ANDREA FERNA...	🔥 Medium	LISTO
☐	PIED1-26 Configurar intervalo entre citas	SG SARAH ANDREA QUI...	AS ANDREA FERNA...	🔥 Medium	LISTO
☐	PIED1-25 Configurar franjas horarias	SG SARAH ANDREA QUI...	AS ANDREA FERNA...	🔥 Medium	LISTO
☐	PIED1-24 Configurar días de atención	CD CARLOS ANDRES CA...	AS ANDREA FERNA...	🔥 Medium	LISTO
☐	PIED1-23 Configurar semanas habilitadas	AS ANDREA FERNANDA ...	AS ANDREA FERNA...	🔥 Medium	LISTO

+ Crear

29 de 29

↺

Este sprint no solo cubrió la configuración de parámetros de citas y pruebas unitarias, sino que también permitió validar atributos de calidad como escalabilidad (configuración de intervalos y franjas horarias) y seguridad (control de usuarios y roles).

<https://unicauca-team-aaatxnj8.atlassian.net/jira/software/projects/PIED1/list?jql=project+%3D+%22PIED1%22+AND+statusCategory+%3D+Done+AND+statusCategory+ChangedDate+%3E%3D+-1w&atlOrigin=eyJpIjoiZGY5NzEzZWYwYjUwNDc3MmJmOTRlZiMzZGE3MWRiMzQlLCJwIjoiajJ9>

Planificación De Tareas Del Sprint 3

Sistema Piedrazul

Q Search work

I

AS

CD

SG

Filter

Group

	Work	Assignee	Reporter	Priority	Status	Resolution
☐	> PIED1-1 HU1 - Listar citas médicas por médico y fecha	AS ANDREA FERNANDA ...	AS ANDREA FERNA...	Medium	LISTO ✓	Done
☐	> PIED1-2 HU2 - Crear cita desde WhatsApp	SG SARAH ANDREA QUI...	AS ANDREA FERNA...	Medium	LISTO ✓	Done
☐	> PIED1-3 HU3 - Portal de autogestión y agendamiento ...	CD CARLOS ANDRES CA...	AS ANDREA FERNA...	Medium	LISTO ✓	Done
☐	> PIED1-4 HU4 - Configuración global y por profesional	Unassigned	AS ANDREA FERNA...	Medium	LISTO ✓	Done
☒	PIED1-39 Listar citas por médico y fecha: Imp...	I iikanbonilla	I iikanbonilla	Medium	LISTO ✓	Done
☒	PIED1-40 Agendamiento autónomo web: Flujo seguro...	SG SARAH ANDREA QUI...	SG SARAH ANDREA...	Medium	LISTO ✓	Done
☒	PIED1-41 Configuración de parámetros del sistema: D...	CD CARLOS ANDRES CA...	CD CARLOS ANDRE...	Medium	LISTO ✓	Done
☒	PIED1-42 Exportar citas a CSV: Generar archivo com...	SG JUAN SEBASTIAN C...	SG JUAN SEBASTI...	Medium	LISTO ✓	Done
☒	PIED1-43 Re-agendamiento de citas: Permitir cambio...	AS ANDREA FERNANDA ...	AS ANDREA FERNA...	Medium	LISTO ✓	Done

Este sprint cubrió la finalidad del proyecto abarcando la exportación de CSV, Re-Agendamiento, temas de seguridad y demas parametros pedidos por los requerimientos funcionales

[Sistema Piedrazul - Backlog - Jira](#)

ESCENARIOS DE CALIDAD

Escenario 1 - Modificabilidad

Descripción breve: El sistema Piedrazul requiere flexibilidad para alterar las reglas de generación de slots temporales. Esto incluye la definición de citas con duraciones específicas (ej. 45 minutos) en días determinados o la restricción de franjas horarias (bloqueos de mediodía) para profesionales de la salud particulares.

Estímulo: El Administrador solicita la integración de un nuevo algoritmo de disponibilidad sin comprometer la continuidad del servicio de agendamiento.

Respuesta: Se desarrolla una nueva implementación `FridayExtendedSlotStrategy` que concreta la interfaz `SlotCalculationStrategy` y hereda de

`AbstractSlotCalculationStrategy`. Spring Boot registra automáticamente el componente en la lista de estrategias inyectada en `AvailabilityService`. La arquitectura permite extender la funcionalidad sin alterar el código fuente existente, respetando el principio Open/Closed.

Escenario de calidad aplicado:

- Modificabilidad: Integración de la nueva estrategia en ≤ 1 hora de desarrollo.
- Estabilidad: Mantenimiento de cobertura sin regresiones en `AvailabilityServiceTest` y `StandardSlotCalculationStrategyTest`.
- Independencia: Despliegue aislado afectando únicamente al contenedor `availability-service`.

Resultado esperado: El sistema integra las nuevas reglas de negocio sin impactar el funcionamiento de `booking-service` ni `user-service`. La operación para pacientes y agendadores permanece ininterrumpida durante el despliegue de la actualización.

Escenario 2 - Escalabilidad

Descripción breve: La plataforma Piedrazul soporta el uso concurrente de pacientes, agendadores y administradores. Durante periodos de alta demanda (comienzos de semana o jornadas de salud), la carga de solicitudes de agendamiento y consulta de franjas se incrementa hasta cinco veces por encima del promedio operativo habitual.

Estímulo: Se presenta un pico de solicitudes concurrentes de aproximadamente 500 usuarios intentando gestionar citas simultáneamente en el portal web.

Respuesta: Se emplea una arquitectura de microservicios para distribuir la carga. `availability-service` y `booking-service` escalan horizontalmente mediante el uso de balanceadores de carga. RabbitMQ procesa asíncronamente los eventos de WhatsApp, mitigando cuellos de botella. La segregación de datos en `users_db`, `availability_db` y `booking_db` asegura que el rendimiento de un servicio no afecte la persistencia de los demás.

Escenario de calidad aplicado:

- Rendimiento: Latencia en `GET /availability/slots` ≤ 2 segundos en el percentil 95.

- Escalabilidad: Soporte para 1000 solicitudes concurrentes sin degradación del servicio.
- Disponibilidad: Tiempo de actividad ≥ 99.5 % bajo condiciones de carga extrema.
- Eficiencia: Consumo de recursos de infraestructura (CPU/Memoria) inferior al 80 % por instancia.

Resultado esperado: El sistema preserva su integridad funcional ante fluctuaciones de demanda. La interfaz web y las herramientas de agendamiento operan sin latencia perceptible para el usuario final, mientras que el procesamiento asíncrono de eventos garantiza la sincronización fluida entre canales sin bloqueos.

ARQUITECTURA Y DISEÑO

ITERACIÓN 2

Diagrama de Contexto (Nivel 1 C4)

Muestra los actores externos y su relación con el sistema Piedrazul como caja negra. Los actores identificados son: Paciente (agendamiento web/WhatsApp), Médico/Terapista (consulta de citas), Agendador (gestión manual) y Administrador (configuración). El sistema se comunica con WhatsApp Business API y con el servidor de correo electrónico (SMTP) como sistemas externos.

None

@startuml

title Diagrama de Contexto - Sistema Piedrazul

actor Paciente

actor "Médico/Terapista" as Medico

actor Agendador

actor Administrador

```
rectangle "Sistema de Agendamiento de Citas - Piedrazul" {
    note right
        Permite gestionar citas médicas,
        historias clínicas y usuarios.
    end note
}
```

Paciente --> "Sistema de Agendamiento de Citas - Piedrazul" : Agenda citas / recibe confirmación

Medico --> "Sistema de Agendamiento de Citas - Piedrazul" : Registra historia clínica / consulta citas

Agendador --> "Sistema de Agendamiento de Citas - Piedrazul" : Registra y reprograma citas / obtiene listado

Administrador --> "Sistema de Agendamiento de Citas - Piedrazul" : Administra usuarios / configura parámetros

rectangle "Sistema Externo: WhatsApp" as WhatsApp

rectangle "Sistema Externo: Correo Electrónico" as Email

Agendador --> WhatsApp : Recibe solicitudes de pacientes

Paciente --> Email : Recibe notificación de cita

@enduml

Diagrama de Contexto - Sistema Piedrazul

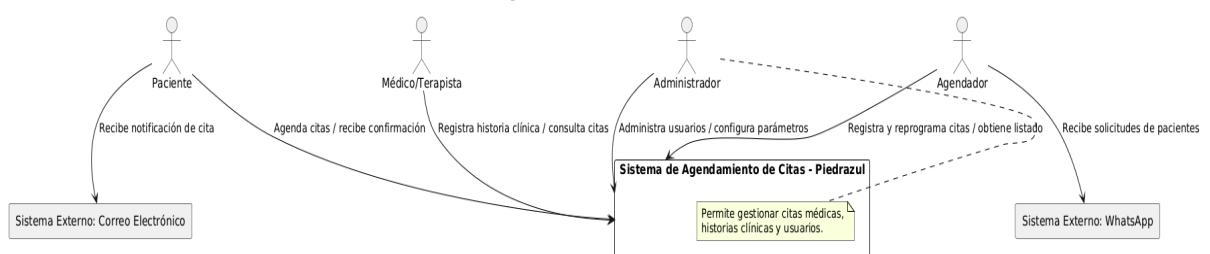


Diagrama de Contenedores (Nivel 2 C4)

Muestra los contenedores que conforman el sistema: Frontend JavaFX, API Gateway (Spring Cloud Gateway), microservicio user-service, microservicio booking-service, microservicio availability-service, broker de mensajes RabbitMQ, y las bases de datos PostgreSQL/MySQL por microservicio. La comunicación entre frontend y backend se realiza mediante HTTP/REST, y la comunicación asíncrona entre microservicios mediante eventos AMQP (RabbitMQ).

None

@startuml

title Diagrama de Contenedores - Sistema Piedrazul

actor Paciente

actor "Médico/Terapista" as Medico

actor Administrador

rectangle "Sistema de Agendamiento de Citas - Piedrazul" {

 rectangle "Frontend Web/Escritorio\n(JavaFX - MVC)" as Frontend

 rectangle "Backend REST API\n(Spring Boot)" as Backend

 database "Base de Datos\nPostgreSQL / MariaDB" as DB

}

Paciente --> Frontend : Usa interfaz web/escritorio

Medico --> Frontend : Usa interfaz web/escritorio

Administrador --> Frontend : Usa interfaz web/escritorio

Frontend --> Backend : Solicitudes HTTP (REST)

Backend --> DB : Lee/Escribe datos (JDBC/JPA)

rectangle "Sistema Externo: WhatsApp" as WhatsApp

rectangle "Sistema Externo: Correo Electrónico" as Email

Backend --> WhatsApp : Procesa solicitudes de citas

Backend --> Email : Envía notificaciones de citas

@enduml

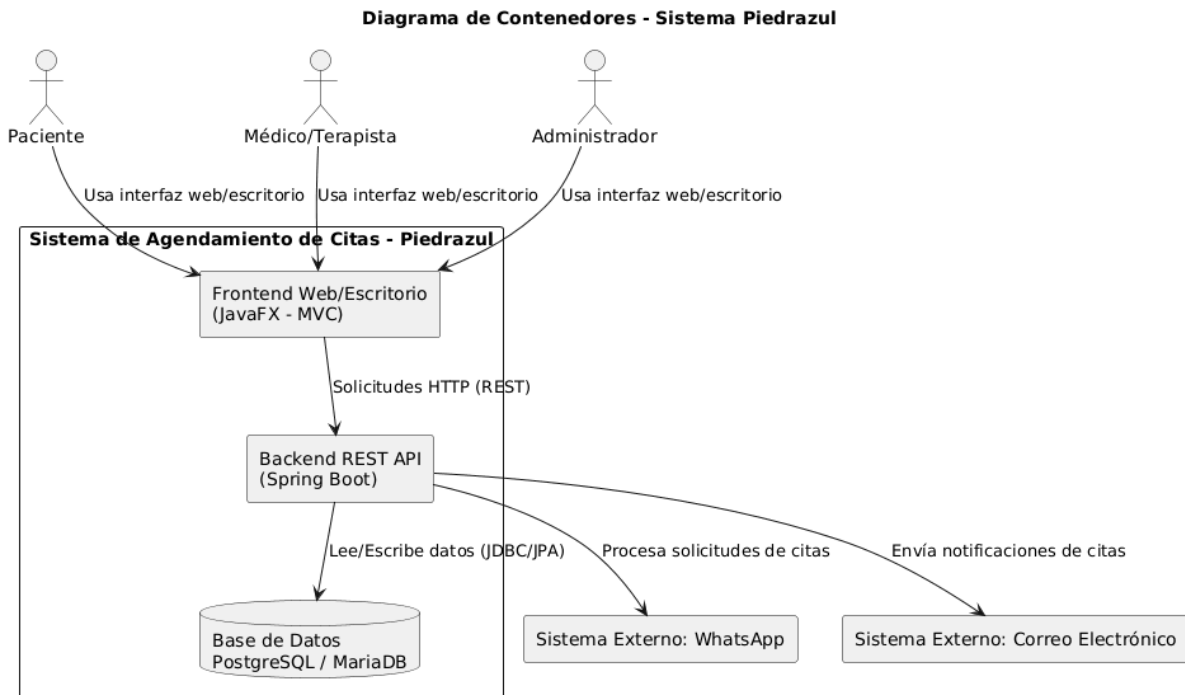


Diagrama de Componentes (Nivel 3 C4)

Detalla los componentes internos de cada microservicio. El booking-service incluye: BookingController, WebBookingFacade, WhatsAppAppointmentListener, AvailabilityClientAdapter y AppointmentRepository. El user-service incluye: UserController, UserService y la entidad User con el método estático de fábrica. El availability-service incluye: AvailabilityController, el Strategy de cálculo de franjas y el repositorio de disponibilidad.

None

@startuml

title Diagrama de Componentes - Sistema Piedrazul (Lógica de Negocio)

package "Sistema Piedrazul - Monolito MVC" {

component "Controllers\n(Spring Controllers)" as Controller

package "Services\n(Lógica de Negocio)" {

component "CitaService" as CitaService

component "UsuarioService" as UsuarioService

component "AuthService" as AuthService

component "AgendaService" as AgendaService

component "ProfesionalService" as ProfesionalService

component "NotificacionService" as NotificacionService

}

```

component "Repositories\n(JPA)" as Repository
component "Security Module\n(Spring Security)" as Security
}

```

```

database "Base de Datos\nPostgreSQL / MariaDB" as DB

```

```

Controller --> CitaService : Gestiona citas
Controller --> UsuarioService : Gestiona usuarios
Controller --> AuthService : Autenticación
Controller --> AgendaService : Maneja agenda
Controller --> ProfesionalService : Gestiona profesionales
Controller --> NotificacionService : Envía notificaciones

```

```

CitaService --> Repository : CRUD citas
UsuarioService --> Repository : CRUD usuarios
AgendaService --> Repository : CRUD agenda
ProfesionalService --> Repository : CRUD profesionales
NotificacionService --> Repository : CRUD notificaciones

```

```

AuthService --> Security : Valida credenciales y roles
Repository --> DB : Lee/Escribe datos

```

```

@enduml

```

Diagrama de Componentes - Sistema Piedrazul (Lógica de Negocio)

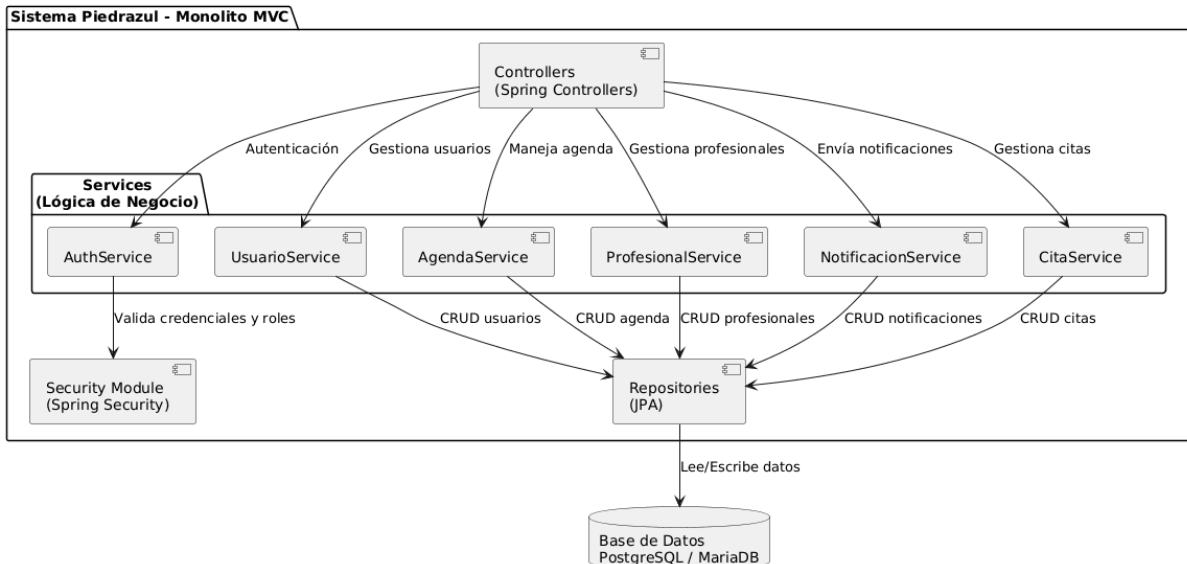


Diagrama de Bounded Contexts

El sistema se divide en cuatro contextos acotados (Bounded Contexts): Contexto de Agendamiento (booking-service), Contexto de Usuarios y Autenticación (user-service), Contexto de Disponibilidad (availability-service) y Contexto de Configuración. Cada contexto tiene su propio modelo de dominio y se comunican

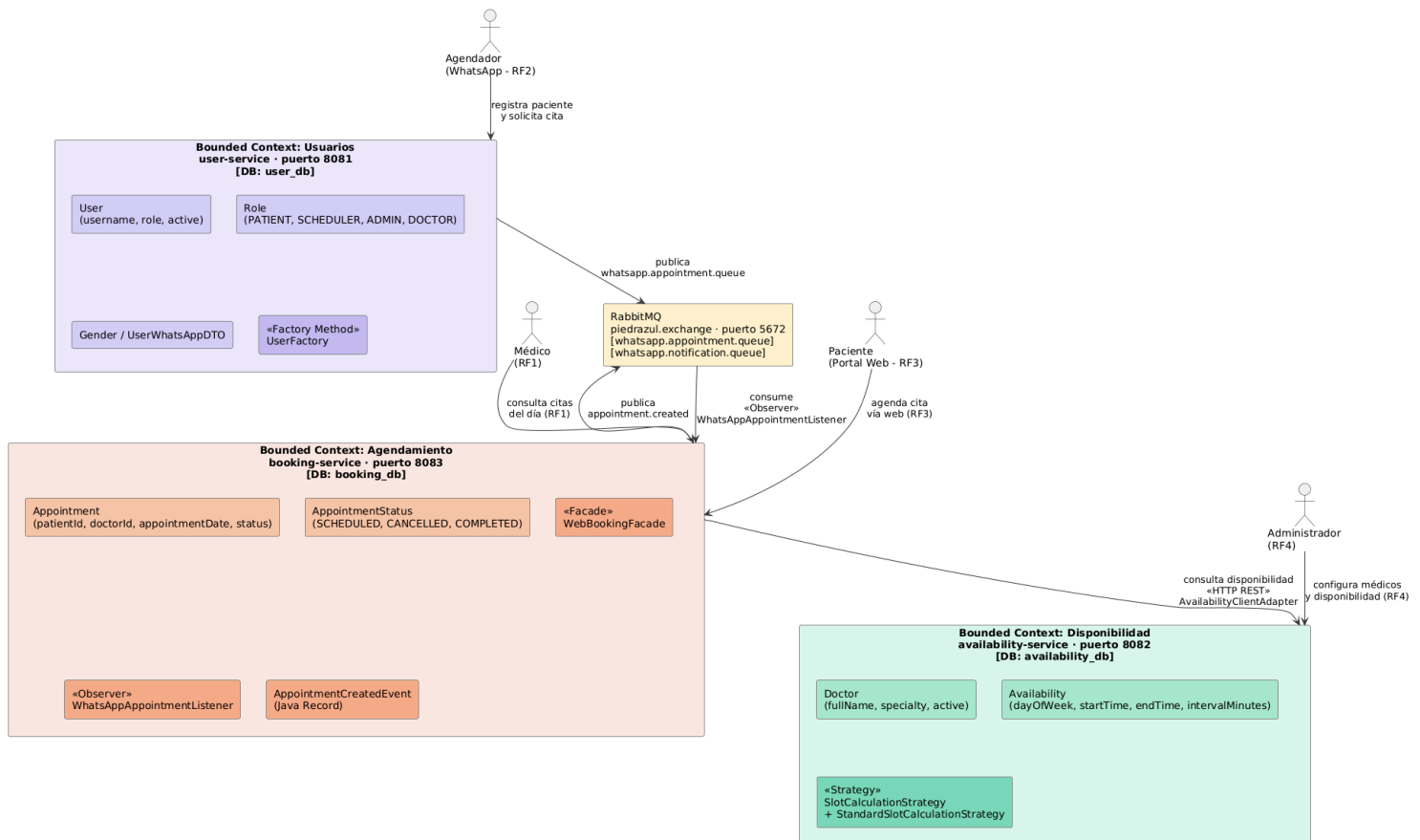


Diagrama de Clases UML

El modelo de dominio incluye las entidades principales: User (con roles PATIENT, DOCTOR, SCHEDULER, ADMIN), Doctor (extiende User), Appointment (cita agendada con estado PENDING/CONFIRMED/CANCELLED), Availability (disponibilidad de un médico en un slot), y los DTOs para la comunicación entre microservicios. Todas las entidades utilizan la anotación @Builder de Lombok.

```
None
@startuml
title Diagrama de Clases - Sistema Piedrazul

class Usuario {
    id : Long
    username : String
```

```
    password : String
    estado : Boolean
}
```

```
class Rol {
    id : Long
    nombre : String
}
```

```
class Paciente {
    id : Long
    nombreCompleto : String
    identificacion : String
    correo : String
    telefono : String
}
```

```
class Profesional {
    id : Long
    nombreCompleto : String
    especialidad : String
    intervaloCitas : int
    estado : Boolean
}
```

```
class Cita {
    id : Long
    fecha : Date
    hora : String
    estado : String
}
```

```
class HistoriaClinica {
    id : Long
    fecha : Date
    descripcion : String
}
```

```
class Auditoria {
    id : Long
    accion : String
    fechaHora : Date
}
```

Usuario --> Rol : Tiene

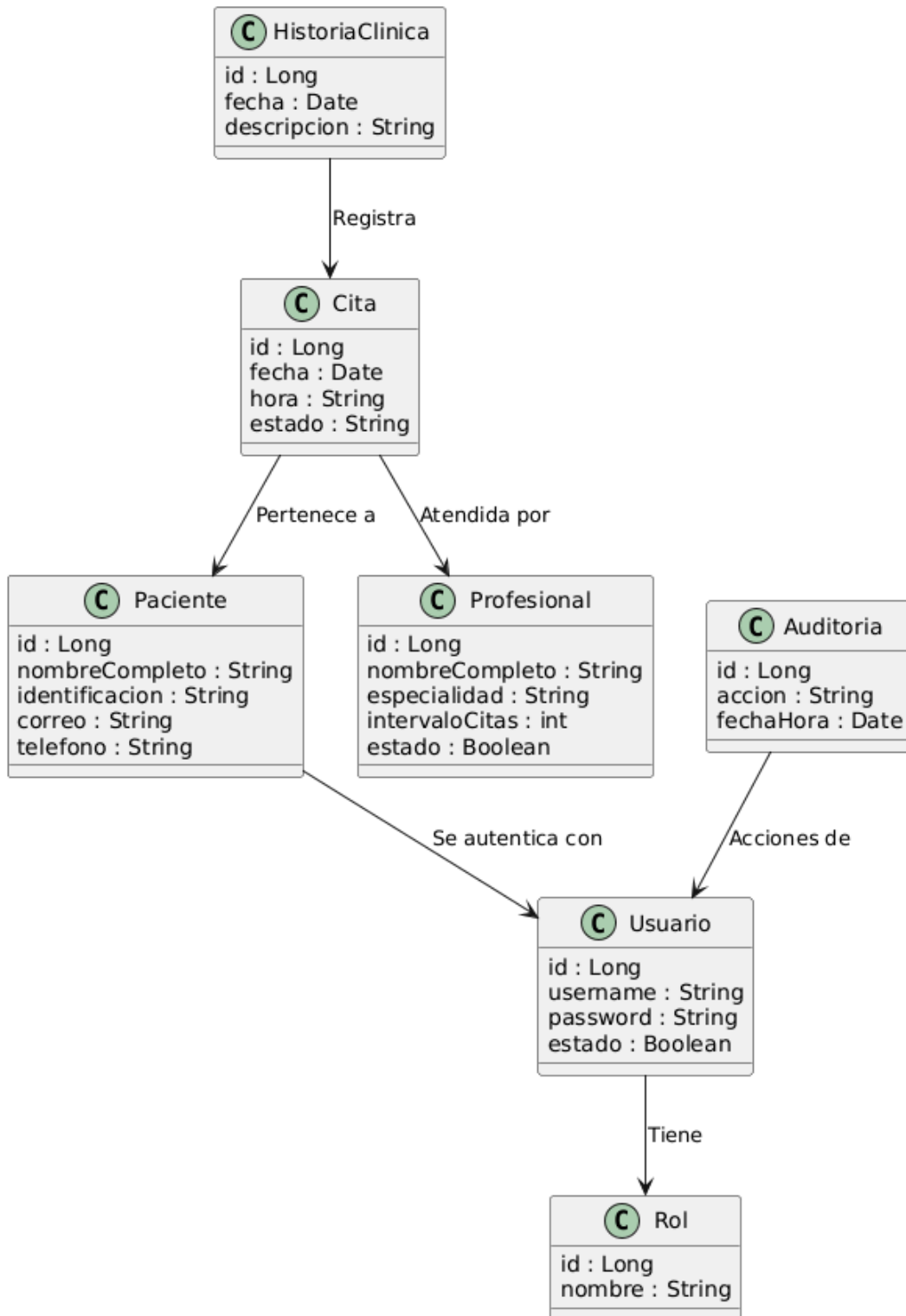
Paciente --> Usuario : Se autentica con

Cita --> Paciente : Pertenece a

Cita --> Profesional : Atendida por

HistoriaClinica --> Cita : Registra
Auditoria --> Usuario : Acciones de
@enduml

Diagrama de Clases - Sistema Piedrazul



ITERACIÓN 3

Diagrama de Contexto (Nivel 1 C4)

Muestra la evolución del sistema en el tercer corte: los mismos actores externos ahora utilizan capacidades ampliadas del sistema. Paciente realiza registro y agendamiento autónomo vía web; Agendador gestiona citas WhatsApp y exporta listados CSV; Médico/Terapista re-agenda citas existentes conservando historial; Administrador configura la ventana de agendamiento, días de atención, franjas horarias e intervalos por profesional. El sistema sigue recibiendo solicitudes originadas en WhatsApp (canal del agendador), pero el paciente ya no depende exclusivamente de ese canal para reservar.

```
None
@startuml
title Diagrama de Contexto - Iteracion 3 - Sistema Piedrazul

actor Paciente
actor Agendador
actor "Medico/Terapista" as Medico
actor Administrador

rectangle "Sistema de Reserva de Citas - Piedrazul" {
    note right
        Agendamiento web autonomo,
        configuracion admin, CSV,
        re-agendamiento con historial.
    end note
}

Paciente --> "Sistema de Reserva de Citas - Piedrazul" : Registro +
agendamiento web
Agendador --> "Sistema de Reserva de Citas - Piedrazul" : Citas WhatsApp +
export CSV
Medico --> "Sistema de Reserva de Citas - Piedrazul" : Re-agenda citas +
consulta historial
Administrador --> "Sistema de Reserva de Citas - Piedrazul" : Configura ventana
y disponibilidad

@enduml
```

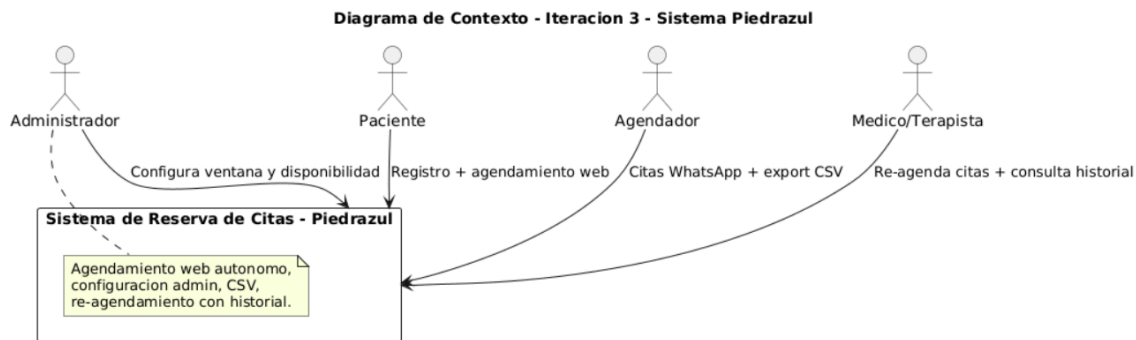



Diagrama de Contenedores (Nivel 2 C4)

Muestra los contenedores de la iteración 3 con las nuevas rutas funcionales del portal: /agendador, /paciente y /admin. El Portal Angular 21 consume tres APIs REST independientes. user-service gestiona registro web y WhatsApp; availability-service expone franjas disponibles y configuración global (system-config); booking-service centraliza creación de citas, agendamiento web (web-booking), exportación CSV y re-agendamiento con historial. RabbitMQ mantiene el desacoplamiento asíncrono; las tres bases PostgreSQL garantizan aislamiento de datos por servicio. La capa de seguridad transversal incluye CORS restrictivo, validación Jakarta en DTOs y manejo centralizado de excepciones sin exposición de stack traces.

None

```
@startuml
title Diagrama de Contenedores - Iteracion 3 - Sistema Piedrazul
actor Paciente
actor Agendador
actor Medico
actor Administrador
rectangle "Sistema Piedrazul" {
    rectangle "Portal Angular 21" as Frontend {
        portin "/agendador"
        portin "/paciente"
        portin "/admin"
    }
    rectangle "user-service :8081" as UserSvc
    rectangle "availability-service :8082" as AvailSvc
    rectangle "booking-service :8083" as BookSvc
    queue "RabbitMQ" as MQ
    database "users_db" as UsersDB
    database "availability_db" as AvailDB
    database "booking_db" as BookDB
}
```

```

Paciente --> Frontend : /paciente
Agendador --> Frontend : /agendador
Administrador --> Frontend : /admin/disponibilidad
Medico --> Frontend : /agendador (re-agendar)
Frontend --> UserSvc : web-register
Frontend --> AvailSvc : slots + system-config
Frontend --> BookSvc : web-booking, CSV, reschedule
UserSvc --> UsersDB
AvailSvc --> AvailDB
BookSvc --> BookDB
UserSvc --> MQ
MQ --> BookSvc
note as N1
  **Seguridad transversal:**
  - CORS solo localhost:4200
  - @Valid en DTOs (Jakarta)
  - GlobalExceptionHandler sin stack traces
  - Unicidad documento/email/telefono
  - State pattern bloquea re-agendar citas terminales
end note
@enduml

```

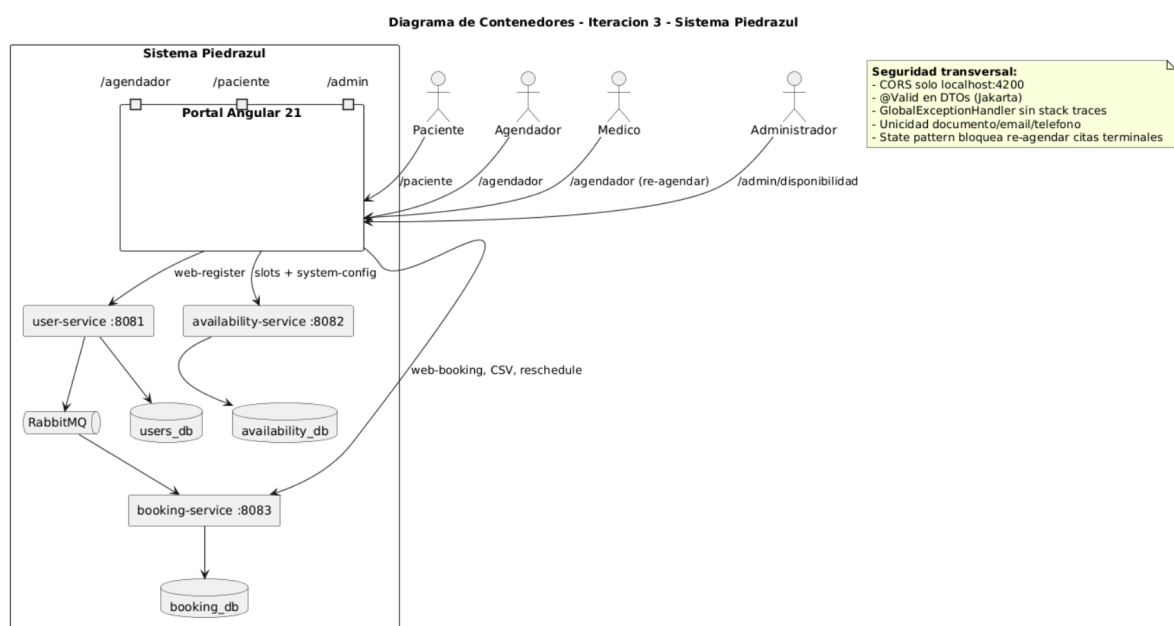


Diagrama de Componentes (Nivel 3 C4) - booking-service

Detalla los componentes internos del microservicio de citas en la iteración 3. AppointmentController expone los endpoints de creación, listado, agendamiento web, exportación CSV y re-agendamiento. WebBookingFacade (patrón Facade) orquesta la verificación de disponibilidad y la creación de la cita web en una sola

operación. `CommandInvoker` y `RescheduleAppointmentCommandHandler` (patrón `Command`) encapsulan el re-agendamiento; `AppointmentStateContext` (patrón `State`) impide operaciones sobre citas en estados terminales (cancelada/completada). `AvailabilityClientAdapter` y `UserClientAdapter` integran otros microservicios; `WhatsAppAppointmentListener` y `AppointmentCreatedNotificationListener` (patrón `Observer` vía `RabbitMQ`) procesan eventos asíncronos. `GlobalExceptionHandler` normaliza errores de forma segura; `AppointmentRescheduleHistoryRepository` persiste el historial exigido por el requisito 6.

None

```
@startuml
title Diagrama de Componentes - Iteracion 3 - booking-service

package "booking-service" {

    component "AppointmentController" as ApptCtrl

    component "WebBookingFacade\n(Facade)" as Facade
    component "AppointmentService" as ApptSvc

    component "CommandInvoker\n(Command)" as Invoker
    component "RescheduleAppointmentCommandHandler" as RescheduleHandler
    component "RescheduleAppointmentCommand" as RescheduleCmd

    component "AppointmentStateContext\n(State)" as StateCtx
    component "PendingAppointmentState" as Pending
    component "TerminalAppointmentState" as Terminal

    component "AvailabilityClientAdapter\n(Adapter)" as AvailAdapter
    component "UserClientAdapter\n(Adapter)" as UserAdapter

    component "WhatsAppAppointmentListener\n(Observer)" as WaListener
    component "AppointmentCreatedNotificationListener\n(Observer)" as
NotifListener

    component "GlobalExceptionHandler\n(Seguridad)" as ExHandler
    component "AppointmentRepository" as ApptRepo
    component "AppointmentRescheduleHistoryRepository" as HistRepo
}

database "booking_db" as DB
queue "RabbitMQ" as MQ

ApptCtrl --> Facade : web-booking
ApptCtrl --> ApptSvc : CRUD, CSV, reschedule
ApptCtrl --> ExHandler : Errores seguros
```

```
Facade --> AvailAdapter : Verificar franjas
Facade --> ApptSvc : Crear cita
```

```
ApptSvc --> Invoker
Invoker --> RescheduleHandler
RescheduleHandler --> RescheduleCmd
RescheduleHandler --> StateCtx
RescheduleHandler --> AvailAdapter
RescheduleHandler --> HistRepo
StateCtx --> Pending
StateCtx --> Terminal
```

```
ApptSvc --> ApptRepo
ApptSvc --> UserAdapter
ApptRepo --> DB
HistRepo --> DB
```

```
MQ --> WaListener
WaListener --> ApptSvc
ApptSvc --> MQ : Notifica cita creada
MQ --> NotifListener
```

```
note right of StateCtx
  **Seguridad de negocio:**
  CANCELLED y COMPLETED
  no permiten re-agendamiento
end note
```

```
note right of Facade
  **Seguridad de integridad:**
  Valida franja en servidor
  antes de persistir cita web
end note
@enduml
```

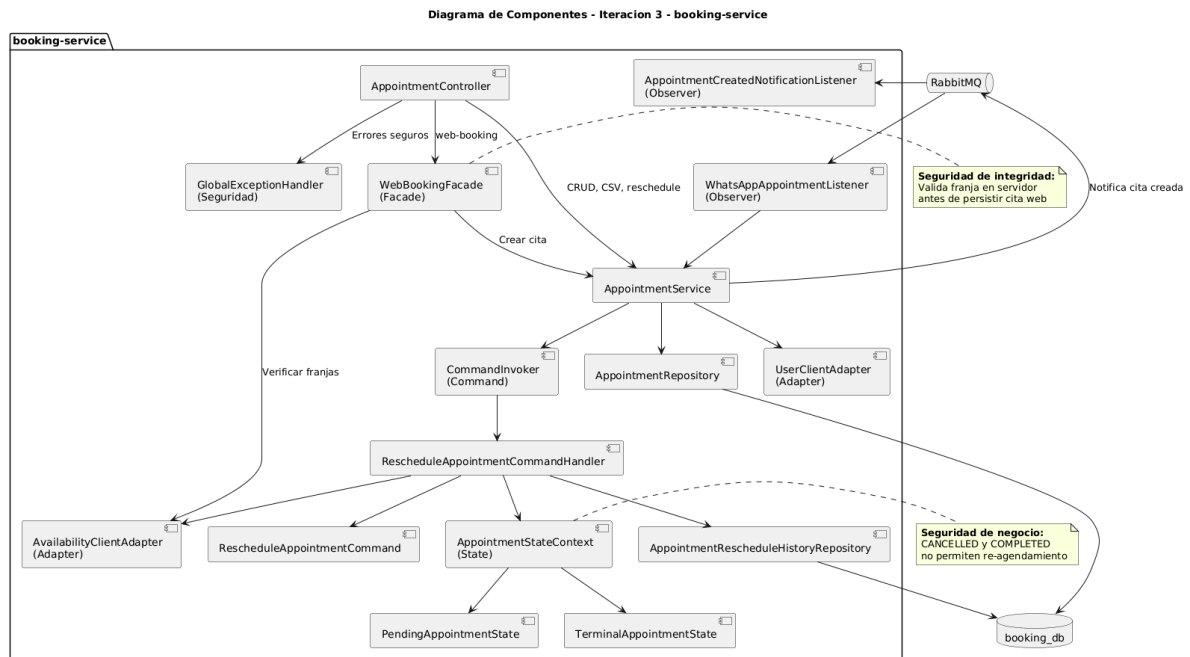


Diagrama de Bounded Contexts

Descripción breve: El sistema se divide en tres contextos acotados alineados a los microservicios: Usuarios y Pacientes (user-service), Disponibilidad y Configuración (availability-service) y Agendamiento de Citas (booking-service). Cada contexto tiene su propio modelo de dominio y base de datos PostgreSQL. Se comunican mediante llamadas REST sincrónicas (patrón Adapter) y eventos asíncronos en RabbitMQ cuando un paciente contactado por WhatsApp debe generar una cita.

None

@startuml

title Diagrama de Bounded Contexts - Sistema Piedrazul

```

skinparam rectangle {
    BorderColor #333333
    BackgroundColor #F5F5F5
}

```

```

skinparam database {
    BorderColor #333333
    BackgroundColor #E8F4FD
}

```

```

skinparam queue {
    BorderColor #333333
    BackgroundColor #FFF3E0
}

rectangle "Contexto: Usuarios y Pacientes\n(user-service :8081)" as UserCtx {
    component "UserService" as UserSvc
    component "UserController" as UserCtrl
    database "users_db\n(PostgreSQL)" as UsersDB

    note bottom of UserCtx
        **Agregado principal:** User
        **Enums:** Role, Gender
        **Factory Method:** createPatientFromWhatsApp()
    end note

    UserCtrl --> UserSvc
    UserSvc --> UsersDB
}

rectangle "Contexto: Disponibilidad y Configuracion\n(availability-service :8082)" as AvailCtx {
    component "AvailabilityService" as AvailSvc
    component "DoctorService" as DocSvc
    component "SystemConfigService" as ConfigSvc
    component "SlotCalculationStrategy" as Strategy
    database "availability_db\n(PostgreSQL)" as AvailDB

    note bottom of AvailCtx
        **Agregados:** Doctor, Availability
        **Config global:** SystemConfig
        **Patron:** Strategy + Template Method
    end note

    AvailSvc --> Strategy
    AvailSvc --> AvailDB
    DocSvc --> AvailDB
    ConfigSvc --> AvailDB
}

rectangle "Contexto: Agendamiento de Citas\n(booking-service :8083)" as BookCtx {
    component "AppointmentService" as ApptSvc
    component "WebBookingFacade" as Facade
    component "CommandInvoker" as Invoker
    component "AppointmentStateContext" as StateCtx
    database "booking_db\n(PostgreSQL)" as BookDB
}

```

note bottom of BookCtx

****Agregados:**** Appointment

****Historial:**** AppointmentRescheduleHistory

****Patrones:**** Facade, Command, State, Observer

end note

Facade --> ApptSvc

Invoker --> ApptSvc

ApptSvc --> StateCtx

ApptSvc --> BookDB

}

queue "RabbitMQ\nwhatsapp.appointment.queue" as MQ

' Comunicacion entre contextos

UserCtx --> MQ : Publica evento\n(UserWhatsAppDTO)

MQ --> BookCtx : WhatsAppAppointmentListener\n(Observer)

BookCtx --> AvailCtx : REST\nGET /availability/slots\nGET /occupied-times

AvailCtx --> BookCtx : REST\nGET /appointments/doctor/{id}/occupied-times

BookCtx --> UserCtx : REST\nGET /users/{id}

legend right

|= Tipo |= Mecanismo |

| Sincrono | REST via Adapters |

| Asincrono | RabbitMQ (AMQP) |

| Referencia cruzada | patientId, doctorId (IDs, no FK entre BD) |

endlegend

@enduml

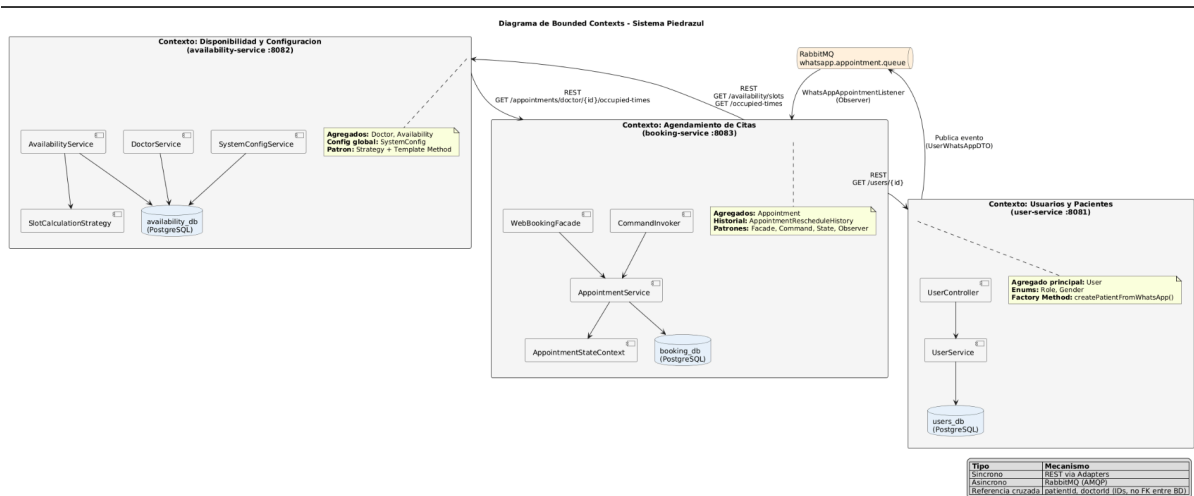


Diagrama de Clases UML

Descripción breve: Modelo de dominio distribuido en tres microservicios. En user-service: User con roles y datos demográficos del paciente. En availability-service: Doctor, Availability (franja por día) y SystemConfig (ventana de agendamiento). En booking-service: Appointment (referencias por ID a paciente y médico) y AppointmentRescheduleHistory para el requisito 6. Los DTOs facilitan la comunicación entre capas y servicios.

None

@startuml

title Diagrama de Clases - Sistema Piedrazul (Modelo de Dominio)

skinparam classAttributeIconSize 0

skinparam packageStyle rectangle

' ===== USER-SERVICE =====

package "user-service (users_db)" #E8F5E9 {

class User {

- id : Long

- username : String

- password : String

- fullName : String

- documentNumber : String

- phone : String

- email : String

- gender : Gender

- birthDate : LocalDate

- role : Role

- active : boolean

- doctorId : Long

+ {static} createPatientFromWhatsApp(...) : User

}

enum Role {

SCHEDULER

PATIENT

DOCTOR

ADMINISTRATOR

}

enum Gender {

MALE

FEMALE

OTHER


```

    }

    class UserWhatsAppDTO <<DTO>> {
        + documentNumber : String
        + firstName : String
        + lastName : String
        + phone : String
        + gender : Gender
        + birthDate : LocalDate
        + email : String
    }

    User --> Role : tiene
    User --> Gender
    User ..> UserWhatsAppDTO : <<crea desde>>
}

' ===== AVAILABILITY-SERVICE =====
package "availability-service (availability_db)" #E3F2FD {

    class Doctor {
        - id : Long
        - fullName : String
        - specialty : String
        - active : boolean
    }

    class Availability {
        - id : Long
        - dayOfWeek : DayOfWeek
        - startTime : LocalTime
        - endTime : LocalTime
        - intervalMinutes : Integer
        - active : boolean
    }

    class SystemConfig {
        - id : Long
        - bookingWindowWeeks : int
    }

    interface SlotCalculationStrategy {
        + calculateAvailableSlots(schedule, date) : List<LocalTime>
        + supports(schedule) : boolean
    }

    abstract class AbstractSlotCalculationStrategy {
        + {final} calculateAvailableSlots(...)
    }
}

```

```

    # generateTimeSlots(...) : List<LocalTime>
}

class StandardSlotCalculationStrategy {
    # generateTimeSlots(...) : List<LocalTime>
}

class AvailabilityRequestDTO <<DTO>> {
    + doctorId : Long
    + dayOfWeek : DayOfWeek
    + startTime : LocalTime
    + endTime : LocalTime
    + intervalMinutes : Integer
}

Doctor "1" -- "*" Availability : configura
StandardSlotCalculationStrategy --|> AbstractSlotCalculationStrategy
AbstractSlotCalculationStrategy ..|> SlotCalculationStrategy
Availability ..> SlotCalculationStrategy : calcula franjas
}

' ===== BOOKING-SERVICE =====
package "booking-service (booking_db)" #FFF3E0 {

    class Appointment {
        - id : Long
        - patientId : Long
        - doctorId : Long
        - appointmentDate : LocalDateTime
        - status : AppointmentStatus
        - whatsappNumber : String
        - notes : String
    }

    enum AppointmentStatus {
        PENDING
        CONFIRMED
        CANCELLED
        COMPLETED
        RESCHEDULED
    }

    class AppointmentRescheduleHistory {
        - id : Long
        - appointmentId : Long
        - previousDate : LocalDateTime
        - newDate : LocalDateTime
        - responsibleUserId : Long
    }
}

```

```

    - responsibleName : String
    - changedAt : LocalDateTime
}

class WebBookingRequestDTO <<DTO>> {
    + patientId : Long
    + doctorId : Long
    + appointmentDate : LocalDate
    + appointmentTime : LocalTime
    + notes : String
}

class RescheduleRequestDTO <<DTO>> {
    + newAppointmentDate : LocalDateTime
    + responsibleUserId : Long
    + responsibleName : String
}

interface AppointmentState {
    + ensureCanReschedule() : void
}

class AppointmentStateContext {
    - states : Map<AppointmentStatus, AppointmentState>
    + ensureCanReschedule(status) : void
}

class PendingAppointmentState
class TerminalAppointmentState
class RescheduledAppointmentState

Appointment --> AppointmentStatus
Appointment "1" -- "*" AppointmentRescheduleHistory : historial
AppointmentStateContext --> AppointmentState
PendingAppointmentState ..|> AppointmentState
TerminalAppointmentState ..|> AppointmentState
RescheduledAppointmentState ..|> AppointmentState
}

' ===== RELACIONES ENTRE CONTEXTOS (por ID, no FK) =====
Appointment ..> User : patientId >>
Appointment ..> Doctor : doctorId >>

note as N1
    **Referencias entre contextos:**
    patientId y doctorId son Long (IDs logicos).
    No hay FK entre bases de datos distintas.
    La integracion se resuelve via REST Adapters.

```


Aspectos de seguridad evidenciados en C4

Aspecto	Implementación	Ubicación
Validación de entrada	@Valid, @NotBlank, @NotNull en DTOs	Controllers de los 3 servicios
CORS restrictivo	Solo http://localhost:4200	CorsConfig en cada servicio
Contraseñas seguras	UUID auto-generado para pacientes	User.createPatientFromWhatsApp()
Errores controlados	JSON {error, message} sin stack trace	GlobalExceptionHandler
Integridad de datos	Unicidad en documento, email, teléfono	Entidad User + JPA constraints
Reglas de negocio	State pattern bloquea operaciones inválidas	AppointmentStateContext
Anti doble-reserva	Validación de solapamiento + franja disponible	WebBookingFacade, RescheduleAppointmentCommandHandler
Aislamiento de datos	BD por microservicio	users_db, availability_db, booking_db

PATRONES DE DISEÑO GOF

Listado De Patrones De Diseño Implementados

Patrón	Tipo	Clase(s)	Contexto del problema	Cómo se aplicó
Strategy	Comportamiento	SlotCalculationStrategy, StandardSlotCalculationStrategy	Reglas de franjas variables por médico/día	AvailabilityService selecciona la estrategia aplicable e inyecta todas las implementaciones vía Spring
Template Method	Comportamiento	AbstractSlotCalculationStrategy	Evitar duplicar la estructura del algoritmo de slots	Define el esqueleto (validate → generate); subclasses implementan generateTimeSlots()
Facade	Estructural	WebBookingFacade	Agendamiento web requiere verificar disponibilidad y crear cita	Oculta la interacción entre AvailabilityClientAdapter y AppointmentService en un solo método processWebBooking()
Adapter	Estructural	AvailabilityClientAdapter, UserClientAdapter, BookingClientAdapter	Microservicios con APIs REST distintas	Adaptan llamadas HTTP entre servicios para que la capa de aplicación use interfaces locales

Factory Method	Creación	User.createPatientFromWhatsApp()	Pacientes llegan por WhatsApp/web con datos parciales	Centraliza construcción con rol PATIENT, username=documento y password UUID
Observer	Comportamiento	UserService → RabbitMQ → WhatsAppAppointmentListener, AppointmentCreatedNotificationListener	Desacoplar registro de paciente de creación de cita	UserService publica evento; booking-service reacciona asíncronamente
Command	Comportamiento	RescheduleAppointmentCommand, CommandInvoker, RescheduleAppointmentCommandHandler	Re-agendamiento con reglas extensibles	Encapsula la operación; CommandInvoker delega al handler correcto
State	Comportamiento	AppointmentStateContext, PendingAppointmentState, TerminalAppointmentState	Transiciones válidas según estado de cita	Impide re-agendar citas CANCELLED/COMPLETED mediante ensureCanReschedule()

Patrón arquitectónico adicional: Microservicios — base de datos por servicio, comunicación REST síncrona (Adapter) y AMQP asíncrona (Observer), alineado a bounded contexts de Usuarios, Disponibilidad y Citas.

REPOSITORIO GIT

- [andresdaza13/piedrazul-system at feature/final_proyect](#)